

Boosting II (XGBoost)

Amartya Sanyal

`amsa@di.ku.dk`

Department of Computer Science
University of Copenhagen

Based on slides by Fabian Giesecke.

Today!

Image source: <https://dnc1994.com/2016/05/rank-10-percent-in-first-kaggle-competition-en/>

[Table of Contents](#)[Overview](#)

Model Selection

When the features are set, we can start training models. Kaggle competitions usually favor **tree-based models**:

- **Gradient Boosted Trees**
- Random Forest
- Extra Randomized Trees

The following models are slightly worse in terms of general performance, but are suitable as base models in ensemble learning (will be discussed later):

- SVM
- Linear Regression
- Logistic Regression
- Neural Networks

Note that this does not apply to computer vision competitions which are pretty much dominated by neural network models.

All these models are implemented in **Sklearn**.

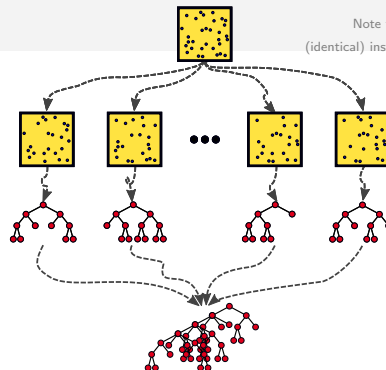
Here I want to emphasize the greatness of **Xgboost**. The outstanding performance of gradient boosted trees and Xgboost's efficient implementation makes it very popular in Kaggle competitions. Nowadays almost every winner uses Xgboost in one way or another.

Updated on Oct 28th, 2016: Recently Microsoft open sourced **LightGBM**, a potentially better

Outline

- 1 Additive Training
- 2 Simplifying the Objective
- 3 Structure Score & Tree Construction
- 4 Summary

Recap: Bagging

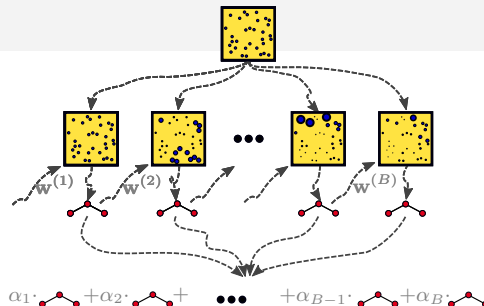


Note that we consider multisets, i.e., we allow multiple (identical) instances in the data T and bootstrap samples S_i .

Bootstrap Aggregating

- 1 Generate B bootstrap samples $S_1, \dots, S_B$ from your training data $T = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\} \subset \mathbb{R}^d \times \mathcal{Y}$ (e.g., by sampling with replacement).
- 2 For each bootstrap sample S_b , build learner/model $\mathcal{T}_b$ (e.g., decision trees).
- 3 The final prediction is obtained by aggregating the predictions made by the learners $\mathcal{T}_1, \dots, \mathcal{T}_B$ (e.g., majority vote for classification).

Recap: Boosting

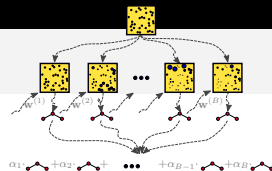


Boosting

- 1 Given training data $T = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\} \subset \mathbb{R}^d \times \mathcal{Y}$ and weights $w_i^{(1)} = \frac{1}{n}$.
- 2 For $b = 1, \dots, B$:
 - Train **weak learner** $\mathcal{T}_b$ based on training data T and weights $\mathbf{w}^{(b)}$.
(e.g., construct i.i.d examples according to $\mathbf{w}^{(b)}$ or incorporate weights into fitting)
 - Increase weights $w_i^{(b)}$ for instances misclassified by $\mathcal{T}_b$; decrease weights $w_i^{(b)}$ for correctly classified instances. Normalize weights; this yields a new vector $\mathbf{w}^{(b+1)}$.
- 3 The final prediction is obtained via a **weighted aggregation of the predictions** made by the learners $\mathcal{T}_1, \dots, \mathcal{T}_B$ (e.g., weighted majority vote).

weak learner = model slightly better than guessing (e.g., decision stump)

Recap: AdaBoost



AdaBoost Algorithm

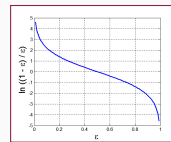
Require: Let $T = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\} \subset \mathbb{R}^d \times \{-1, 1\}$ and $B \in \mathbb{N}$

- 1: $\mathbf{w}^{(1)} = (\frac{1}{n}, \dots, \frac{1}{n}) \in \mathbb{R}^n$
- 2: **for** $b = 1, \dots, B$ **do**
- 3: $h_b = \text{FITWEAKLEARNER}(T, \mathbf{w}^{(b)})$
- 4: Compute weighted classification error $\varepsilon_b = \sum_{j=1}^n w_j^{(b)} \mathbb{I}(h_b(\mathbf{x}_j) \neq y_j)$ of h_b
- 5: Ensure $\varepsilon_b < 0.5$ (e.g., by “inverting” classifier)
- 6: Compute importance $\alpha_b = \frac{1}{2} \ln \left(\frac{1-\varepsilon_b}{\varepsilon_b} \right)$ of classifier
- 7: For $i = 1, \dots, n$, update weights via

$$w_i^{(b+1)} = \frac{w_i^{(b)} \exp(-\alpha_b y_i h_b(\mathbf{x}_i))}{\sum_{j=1}^n w_j^{(b)} \exp(-\alpha_b y_j h_b(\mathbf{x}_j))}$$

8: **end for**

Output hypothesis: $h(\mathbf{x}) = \text{sign} \left(\sum_{b=1}^B \alpha_b h_b(\mathbf{x}) \right)$



Outline

- 1 Additive Training
- 2 Simplifying the Objective
- 3 Structure Score & Tree Construction
- 4 Summary

General Idea

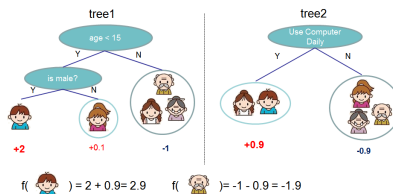
Image source: <https://xgboost.readthedocs.io>

XGBoost

- Training data $\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\} \in \mathbb{R}^d \times \mathbb{R}$
- The final model $\hat{y}_i = \sum_{k=1}^K f_k(\mathbf{x}_i)$ is based on K trees $f_k \in \mathcal{F}$, where $\mathcal{F}$ is the space of decision trees.
- **Learning goal:** Find K such trees that minimize the objective

$$\sum_{i=1}^n \mathcal{L}(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

where $\mathcal{L}$ is a **loss function** and $\Omega(f_k)$ a measure for the **complexity** of a tree.



Original paper ([CG16]): Tianqi Chen and Carlos Guestrin. **XGBoost: A Scalable Tree Boosting System**, KDD, 2016.

Trade-Off: Small Loss vs. Complexity

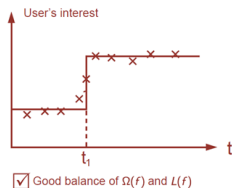
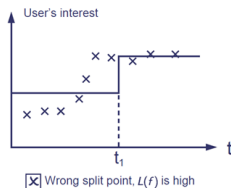
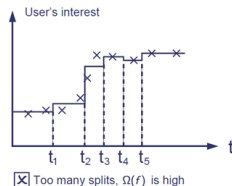
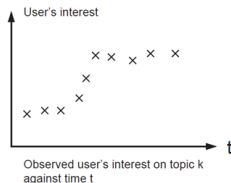


Image source: <https://xgboost.readthedocs.io>

Goal & Training

Objective

Find an ensemble $\hat{y}_i = \sum_{k=1}^K f_k(\mathbf{x}_i)$ of K trees $f_k \in \mathcal{F}$ that minimizes

$$\sum_{i=1}^n \mathcal{L}(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

with $\Omega(f_k)$ specifying the complexity of the tree f_k .

Comment: It is, in general, difficult to find a “global” optimal solution.

Additive Training (Boosting): Start with constant predictions and add a new function (tree) in each iteration:

$$1 \quad \hat{y}_i^{(0)} = 0$$

$$2 \quad \hat{y}_i^{(1)} = f_1(\mathbf{x}_i) = \hat{y}_i^{(0)} + f_1(\mathbf{x}_i)$$

$$3 \quad \hat{y}_i^{(2)} = f_1(\mathbf{x}_i) + f_2(\mathbf{x}_i) = \hat{y}_i^{(1)} + f_2(\mathbf{x}_i)$$

$$4 \quad \dots$$

$$5 \quad \hat{y}_i^{(t)} = \sum_{k=1}^t f_k(\mathbf{x}_i) = \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)$$

Outline

1 Additive Training

2 Simplifying the Objective

3 Structure Score & Tree Construction

4 Summary

Simplifying the Objective I

- At each step t , we have $\hat{y}_i^{(t)} = \sum_{k=1}^t f_k(\mathbf{x}_i) = \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)$
 - Keep functions $\hat{y}_i^{(t-1)}$ that were added in the previous rounds ...
 - Find new function $f_t(\mathbf{x}_i)$ that minimizes the objective!

- We want to minimize

$$\begin{aligned} Obj^{(t)} &= \sum_{i=1}^n \mathcal{L}(y_i, \hat{y}_i^{(t)}) + \sum_{k=1}^t \Omega(f_k) \\ &= \sum_{i=1}^n \mathcal{L}(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) + \text{constant} \end{aligned}$$

- Let us start with the square loss $\mathcal{L}(y, y') = (y - y')^2$. This yields:

$$\begin{aligned} Obj^{(t)} &= \sum_{i=1}^n \left(y_i - (\hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) \right)^2 + \Omega(f_t) + \text{constant} \\ &= \sum_{i=1}^n \left[2(\hat{y}_i^{(t-1)} - y_i)f_t(\mathbf{x}_i) + f_t(\mathbf{x}_i)^2 \right] + \Omega(f_t) + \text{constant} \end{aligned}$$

Simplifying the Objective II

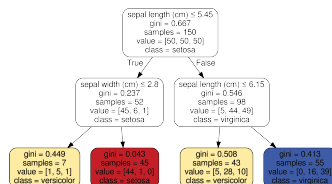
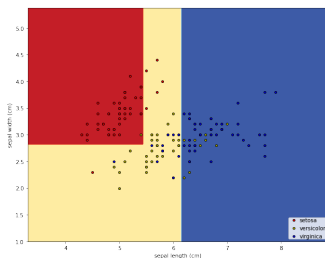
- Omitting the constants, we have the following objective to be minimized:

$$Obj^{(t)} = \sum_{i=1}^n \left[2(\hat{y}_i^{(t-1)} - y_i) f_t(\mathbf{x}_i) + f_t(\mathbf{x}_i)^2 \right] + \Omega(f_t)$$

- Let us define $g_i = 2(\hat{y}_i^{(t-1)} - y_i)$ and $h_i = 2$.
- We can then rewrite the objective in the following way:

$$Obj^{(t)} = \sum_{i=1}^n \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t(\mathbf{x}_i)^2 \right] + \Omega(f_t)$$

Tree Representation



Structure and Leaves

The tree $f_t(\mathbf{x})$ with T leaves can be represented as

$$f_t(\mathbf{x}) = w_q(\mathbf{x}),$$

where $q : \mathbb{R}^d \rightarrow \{1, 2, \dots, T\}$ maps an input vector $\mathbf{x}$ to the **index** of the leaf/region that contains $\mathbf{x}$ and where $\mathbf{w} \in \mathbb{R}^T$ is a vector containing all the **leaf weights**.

Simplifying the Objective III

- We can now **regroup the terms** by the leaves of the tree:

$$\begin{aligned}
 Obj^{(t)} &= \sum_{i=1}^n \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t(\mathbf{x}_i)^2 \right] + \Omega(f_t) \\
 &= \sum_{i=1}^n \left[g_i w_{q(\mathbf{x}_i)} + \frac{1}{2} h_i w_{q(\mathbf{x}_i)}^2 \right] + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \\
 &= \sum_{j=1}^T \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T
 \end{aligned}$$

where $I_j = \{i \mid q(\mathbf{x}_i) = j\}$ and where $\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$ (other regularizers are also possible).

- Thus: The objective can be written as a sum of T independent quadratic functions!

Outline

- 1 Additive Training
- 2 Simplifying the Objective
- 3 Structure Score & Tree Construction**
- 4 Summary

Computing the Structure Score

- By defining $G_j := \sum_{i \in I_j} g_i$ and $H_j := \sum_{i \in I_j} h_i$, we obtain

$$Obj^{(t)} = \sum_{j=1}^T \left[G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma T$$

- For a function of the form $h(v) = Gv + \frac{1}{2}(H + \lambda)v^2$ with $H > 0$, we have

$$\operatorname{argmin}_v h(v) = -\frac{G}{H + \lambda} \quad \text{and} \quad \min_v h(v) = -\frac{1}{2} \frac{G^2}{H + \lambda}$$

- Assuming that the **structure of the tree is fixed**, one can compute the optimal weight in each leaf as well as the overall optimal objective value:

Optimal Weights and Objective Value

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

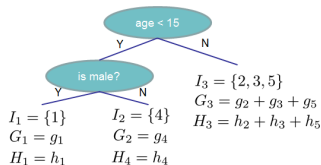
$$Obj^{(t)} = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T$$

XGBoost: Structure Score

Image source: <https://xgboost.readthedocs.io>

Instance index gradient statistics

1		g_1, h_1
2		g_2, h_2
3		g_3, h_3
4		g_4, h_4
5		g_5, h_5



$$Obj = -\sum_j \frac{G_j^2}{H_j + \lambda} + 3\gamma$$

The smaller the score is, the better the structure is

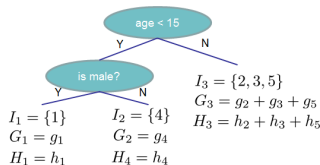
We now have a way to assess the quality of a given tree structure!

Finding the Tree Structure: Approach I

Image source: <https://xgboost.readthedocs.io>

Instance index gradient statistics

1		g_1, h_1
2		g_2, h_2
3		g_3, h_3
4		g_4, h_4
5		g_5, h_5



$$Obj = -\sum_j \frac{G_j^2}{H_j + \lambda} + 3\gamma$$

The smaller the score is, the better the structure is

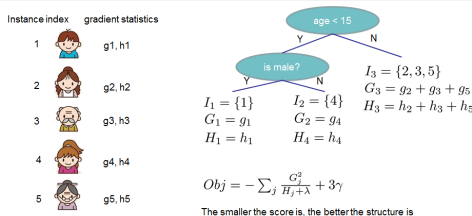
Brute Force

- **Idea:** Enumerate all possible tree structures q !
- For a particular structure: Compute the objective (“quality”) via

$$Obj^{(t)} = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T$$

- Keep track of the structure q with lowest objective ...
- **Problem:** Infinitely many tree structures! Infeasible to check all of them ...

Finding the Tree Structure: Approach II

Image source: <https://xgboost.readthedocs.io>

Greedy Approach

■ Grow the tree in a greedy fashion:

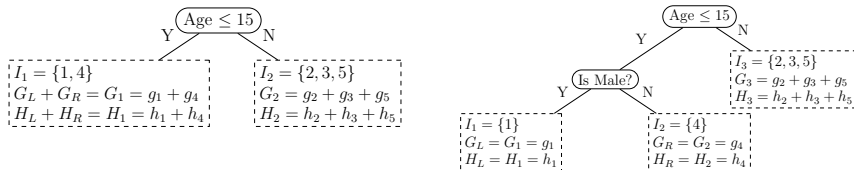
- 1 Start with a tree having depth 0
- 2 At each leaf node, try to split. Use the following “gain” to assess the potential:

$$gain = \frac{1}{2} \left(\underbrace{\frac{G_L^2}{H_L + \lambda}}_{\text{Score of left child}} + \underbrace{\frac{G_R^2}{H_R + \lambda}}_{\text{Score of right child}} - \underbrace{\frac{(G_L + G_R)^2}{H_L + H_R + \lambda}}_{\text{Score of "no split"}} \right) - \underbrace{\gamma}_{\text{Cost for +1 leaf}}$$

- Stop if the best split has negative gain (which can happen).

Finding the Tree Structure: Approach II

$$Obj^{(t)} = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T$$



Greedy Approach

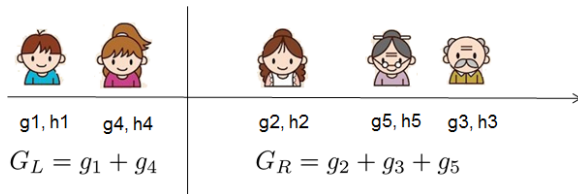
■ Grow the tree in a greedy fashion:

- 1 Start with a tree having depth 0
- 2 At each leaf node, try to split. Use the following “gain” to assess the potential:

$$gain = \frac{1}{2} \left(\underbrace{\frac{G_L^2}{H_L + \lambda}}_{\text{Score of left child}} + \underbrace{\frac{G_R^2}{H_R + \lambda}}_{\text{Score of right child}} - \underbrace{\frac{(G_L + G_R)^2}{H_L + H_R + \lambda}}_{\text{Score of “no split”}} \right) - \underbrace{\gamma}_{\text{Cost for +1 leaf}}$$

- Stop if the best split has negative gain (which can happen).

Finding the Best Split

Image source: <https://xgboost.readthedocs.io>

What is the gain for a split of the form $x_j < a$?

- We need to compute the sum of all g and h values on each side ...
- **Question:** How can one compute all possible split gains efficiently?
- **Observation:** The scores do not change “in between” two data points. We can apply the same approach as for normal decision trees:
 - 1 Pre-sort the instances w.r.t. the j 'th feature.
 - 2 Scan the sorted list from left to right.
 - 3 While scanning, update G_L , H_L , G_R , H_R , and the induced gain.

Defined above: $g_i = 2(\hat{y}_i^{(t-1)} - y_i)$ and $h_i = 2$

Recap: Simplifying the Objective I

- At each step t , we have $\hat{y}_i^{(t)} = \sum_{k=1}^t f_k(\mathbf{x}_i) = \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)$
 - ▶ Keep functions that were added in the previous rounds: $\hat{y}_i^{(t-1)}$
 - ▶ Find new function $f_t(\mathbf{x}_i)$ that minimizes the objective!

- We want to minimize

$$\begin{aligned} Obj^{(t)} &= \sum_{i=1}^n \mathcal{L}(y_i, \hat{y}_i^{(t)}) + \sum_{k=1}^t \Omega(f_k) \\ &= \sum_{i=1}^n \mathcal{L}(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) + \text{constant} \end{aligned}$$

- We have used the square loss $\mathcal{L}(y, y') = (y - y')^2$ so far, which yielded:

$$Obj^{(t)} = \sum_{i=1}^n \left[2(\hat{y}_i^{(t-1)} - y_i) f_t(\mathbf{x}_i) + f_t(\mathbf{x}_i)^2 \right] + \Omega(f_t) + \text{constant}$$

Problem: Other loss functions might not yield such a “nice” form.

Other Loss Functions

■ Objective: $Obj^{(t)} = \sum_{i=1}^n \mathcal{L}(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) + \text{constant}$

■ Idea: Use Taylor series to approximate the loss of each term of the objective!

- Recall: Let f be real function that is infinitely differentiable at a point $a \in \mathbb{R}$, then we have

$$f(x) \approx f(a) + \frac{f'(a)}{1!}(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \dots$$

- Consider $x = a + w$ for some $w \in \mathbb{R}$. Then we have

$$f(a+w) \approx f(a) + \frac{f'(a)}{1!}(w) + \frac{f''(a)}{2!}(w)^2 + \dots$$

■ The labels y_i are fixed. Let us consider the first two terms of the Taylor series to approximate $\mathcal{L}(y_i, \cdot)$ in the point $\hat{y}_i^{(t-1)}$. We need the first and second derivative of $\mathcal{L}(y_i, \cdot)$ in the point $\hat{y}_i^{(t-1)}$:

$$g_i = \left. \frac{d}{dz} \mathcal{L}(y_i, z) \right|_{z=\hat{y}_i^{(t-1)}}$$

$$h_i = \left. \frac{d^2}{dz^2} \mathcal{L}(y_i, z) \right|_{z=\hat{y}_i^{(t-1)}}$$

Other Loss Functions

- Objective: $Obj^{(t)} = \sum_{i=1}^n \mathcal{L}(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) + \text{constant}$
- The labels y_i are fixed. Let us consider the first two terms of the Taylor series to approximate $\mathcal{L}(y_i, \cdot)$ in the point $\hat{y}_i^{(t-1)}$. We need the first and second derivative of $\mathcal{L}(y_i, \cdot)$ in the point $\hat{y}_i^{(t-1)}$:

$$g_i = \left. \frac{d}{dz} \mathcal{L}(y_i, z) \right|_{z=\hat{y}_i^{(t-1)}}$$

$$h_i = \left. \frac{d^2}{dz^2} \mathcal{L}(y_i, z) \right|_{z=\hat{y}_i^{(t-1)}}$$

- Now, we can **approximate** the objective via

$$\begin{aligned} Obj^{(t)} &\approx \sum_{i=1}^n \left[\mathcal{L}(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t(\mathbf{x}_i)^2 \right] + \Omega(f_t) + \text{constant} \\ &= \sum_{i=1}^n \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t(\mathbf{x}_i)^2 \right] + \Omega(f_t) + \text{constant} \end{aligned}$$

- Remaining steps: As before!

Custom Loss Functions

Example

Consider (again) $L(y, y') = (y - y')^2$. Compute:

$$\begin{aligned} g_i &= \left. \frac{d}{dz} \mathcal{L}(y_i, z) \right|_{z=\hat{y}_i^{(t-1)}} \\ h_i &= \left. \frac{d^2}{dz^2} \mathcal{L}(y_i, z) \right|_{z=\hat{y}_i^{(t-1)}} \end{aligned}$$

Example

Consider $L(y, y') = 0.5 \cdot (y - y')^4$. Compute:

$$\begin{aligned} g_i &= \left. \frac{d}{dz} \mathcal{L}(y_i, z) \right|_{z=\hat{y}_i^{(t-1)}} \\ h_i &= \left. \frac{d^2}{dz^2} \mathcal{L}(y_i, z) \right|_{z=\hat{y}_i^{(t-1)}} \end{aligned}$$

XGBoost: In a Nutshell

Objective and Parameters

Find an ensemble $\hat{y}_i = \sum_{k=1}^K f_k(\mathbf{x}_i)$ of K trees $f_k \in \mathcal{F}$ that minimizes

$$\sum_{i=1}^n \mathcal{L}(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

with $\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$.

- K specifies the **number of trees** to be fitted (e.g., 1000 or 10000).
- One can restrict the trees to have a **maximum depths** (e.g., 5).
- One usually specifies a **learning rate** η (default $\eta = 0.3$):

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta \cdot f_t(\mathbf{x}_i)$$

- One can adapt the regularizer via γ and λ .
- One can consider **column-subsampling** (proportion of features to be checked per tree (e.g., 0.3); similar to the `max_features` of Scikit-Learn).
- ...

XGBoost: Implementation

XGBoost

[Docs](#) / XGBoost Documentation

TABLE OF CONTENTS

Installation Guide

Building

Get Star

XGBoost

Frequen

XGBoost

GPU sup

XGBoost

Predicti

XGBoost

Python f

R packa

JVM pac

Ruby pa

Swift package

Julia package

C Package

C++ Interface

CLI Interface

Contribute to XGBoost

XGBoost Documentation

Facts and Ingredients

Efficient implementation for **Gradient Tree Boosting**. *“Among the 29 challenge winning solutions published at Kaggle’s blog during 2015, 17 solutions used XGBoost.”*

- 1 Specialization:** Implementation focuses on solving this problem only.
- 2 Approximation:** Compute approximate quality per split depending on a parameter $\epsilon > 0$ (about $\frac{1}{\epsilon}$ samples considered per split).
- 3 Further hacks:** “Sparsity-aware split finding, column blocks for parallel learning, cache-aware access, blocks for out-of-core computation, compression, ...”

- Distributed XGBoost on Kubernetes
- Distributed XGBoost with XGBoost4j-Spark
- Distributed XGBoost with Dask
- Distributed XGBoost with Ray
- DART booster
- Monotonic Constraints
- Random Forests(TM) in XGBoost
- Feature Interaction Constraints
- Survival Analysis with Accelerated Failure Time
- C API Tutorial
- Text Input Format of DMatrix
- Notes on Parameter Tuning
- Using XGBoost External Memory Version
- Custom Objective and Evaluation Metric

- Frequently Asked Questions
- XGBoost User Forum
- GPU support
- XGBoost Parameters

<https://xgboost.readthedocs.io>

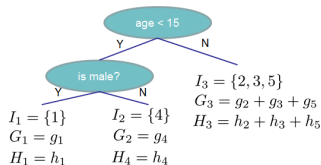
Outline

- 1 Additive Training
- 2 Simplifying the Objective
- 3 Structure Score & Tree Construction
- 4 Summary**

Summary

Instance index gradient statistics

1		g_1, h_1
2		g_2, h_2
3		g_3, h_3
4		g_4, h_4
5		g_5, h_5



$$Obj = -\sum_j \frac{G_j^2}{H_j + \lambda} + 3\gamma$$

The smaller the score is, the better the structure is

References I

[CG16] Tianqi Chen and Carlos Guestrin.

Xgboost: A scalable tree boosting system.

In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, page 785–794, 2016.